

SW 검증 / 김경우

소프트웨어의 동작을 테스트하고, 결과로 확인합니다.

문제를 재현하고 원인을 분석하며, 테스트와 측정으로 개선 결과를 검증해왔습니다.

SELECTED WORK

01 시스템 프로그래밍 과제

셸 · 동시성 서버 · 동적 할당기

02 강의실 예약 백엔드

동시성 제어 · 테스트 · CI

03 학교 축제 서비스 Cardinal

지도 페이지 성능 최적화

— SW 검증 / 김경우

문제를 재현하고 원인을 찾은 뒤, 수정 결과를 다시 확인합니다.

오류가 발생하는 조건을 먼저 확인하고, 수정 전후의 차이를 테스트와 수치로 비교해 왔습니다.

동시 요청에서 발생한 데이터 문제는 테스트로 재현했고, 화면 지연은 직접 측정하며 병목을 좁혔습니다.

시스템 프로그래밍에서는 프로세스, 스레드, 소켓, 메모리 관리 기능을 직접 구현하며 프로그램의 동작 구조를 익혔습니다.

WAY 01

재현부터 시작합니다

간헐적으로 발생하는 문제도 반복해서 확인할 수 있도록 조건을 정리합니다. 동시 요청으로 발생한 데이터 불일치 역시 테스트로 재현한 뒤 수정 전후 결과를 비교했습니다.

강의실 예약 백엔드

Django · PostgreSQL · pytest · GitHub Actions · SonarCloud

WAY 02

수치로 확인합니다

성능 문제는 체감에만 의존하지 않고 지표를 기준으로 확인합니다. 적용한 방법의 효과가 부족하면 다시 측정하고, 병목이 발생하는 지점을 좁혀 해결했습니다.

축제 서비스 Cardinal

React · TypeScript · Chrome DevTools · INP 1130ms → 34ms

WAY 03

동작 구조를 직접 다뤄봤습니다

셸, 동시성 서버, 동적 메모리 할당기를 구현하며 프로세스와 스레드, 소켓 통신, 메모리 관리가 실제로 어떻게 동작하는지 익혔습니다.

시스템 프로그래밍

C · pthread · 소켓 · 시그널 · 셸 · 동시성 서버 · 동적 할당기

셸과 서버, malloc을 C로 구현한 과제입니다.

01 / 시스템 프로그래밍 과제 4건

C · POSIX · 개인 과제 · 자료구조 / 셸 / 서버 / 동적 할당기

한 학기 동안 과제 네 개를 라이브러리 없이 만들었습니다.
모두 요구 사항을 코드로 구현하고 동작을 확인하는 것까지가 범위였습니다.

- 01

MyLib — 자료구조
연결 리스트·해시 테이블·비트맵을 다루고, 명령어 인터프리터로 동작을 확인했습니다
- 02

MyShell — 프로세스
fork · execvp 로 명령을 실행하고 파이프·백그라운드·작업 제어까지 만들었습니다
- 03

Stock Server — 동시성
같은 서버를 select 와 스레드 풀 두 가지로 구현하고 세마포어로 공유 자원을 보호했습니다
- 04

MyMalloc — 메모리
경계 태그와 Segregated free list로 동적 할당기를 만들고 mdriver 로 점수를 확인했습니다

— 과제 01 · 02

자료구조부터 쉘까지 직접 구현했습니다

01

MyLib

Pintos 자료구조

자료구조	함수	핵심 연산
list	26개	이중 연결 리스트 — 정렬·중복 제거·최대/최소 비교 함수 <code>less</code> 를 받아 여러 기준에 활용
hash	16개	버킷별 리스트를 사용하는 체이닝 방식 원소 수에 따라 버킷을 2의 거듭제곱으로 재배치
bitmap	21개	비트 단위 <code>set · flip · scan</code> <code>scan_and_flip</code> 으로 빈 자리를 찾고 바로 표시

직접 작성한 함수

list_swap

값을 바꾸지 않고 앞뒤 포인터를 변경 — 인접한 경우와 떨어진 경우를 나눠 처리

list_shuffle

원소를 꺼낸 뒤 무작위 위치에 다시 삽입하는 방식으로 구현

명령어 처리

create / delete

`strtok`으로 입력을 나누고 명령에 맞는 연산을 실행

실행으로 확인

\$ create list list0

\$ list_sort list0 → 정렬 결과 출력

02

MyShell

직접 만든 쉘

파이프라인 — 여러 프로세스 연결

cat file | grep abc | wc -l

cat
child 1

dup2 → stdout

grep
child 2

dup2 → stdin

wc
child 3

pipe() 로 만든 fd를 dup2 로 표준 입출력에 연결하고, 사용하지 않는 fd는 닫았습니다. 마지막 명령의 결과만 화면에 출력합니다.

시그널 처리

SIGCHLD

waitpid(-1, WNOHANG) 루프로 종료된 자식 프로세스를 회수

SIGINT

Ctrl+C — 포그라운드 작업만 종료하고 목록 갱신

SIGTSTP

Ctrl+Z — 종료하지 않고 정지 상태로 변경

jobs 관리

proc 배열에 pid·명령·상태를 저장해 jobs · bg · fg · kill 구현

포그라운드 / 백그라운드

포그라운드는 sigsuspend 로 SIGCHLD 를 기다려 busy waiting 없이 종료까지 대기합니다.

백그라운드(&)는 바로 다음 입력을 받고, 종료된 프로세스는 핸들러에서 나중에 회수합니다.

PHASE 1

fork · execvp로 명령 실행
cd는 쉘에서 직접 처리

PHASE 2

여러 파이프 연결
다음표 입력 처리

PHASE 3

& 백그라운드 실행
작업 목록 관리

— 과제 03

같은 주식 서버를 두 가지 동시성 모델로 구현했습니다

클라이언트 요청 **show · buy · sell**
종목은 **이진 트리**로 메모리에 보관

두 방식 공통 `load_stock` — 시작 시 파일을 읽어 **트리**로 적재 `buy · sell` — 트리 탐색 후 **수량 갱신** `save_stock` — 종료 시 **파일에 반영**

TASK 1

Event-driven

select()

소켓을 `pool` 에 모아 두고, **읽을 준비가 된 것만** 골라 차례로 처리합니다. 스레드는 **하나**입니다.

구조

client 1
client 2
client 3

select() loop
단일 스레드 · pool 감시

한 번에 **하나씩** 처리하지만, 기다리는 동안 다른 요청을 받을 수 있습니다.

```
pool.nready = Select(pool.maxfd+1,
    &pool.ready_set, NULL, NULL, NULL);

if (FD_ISSET(listenfd, &pool.ready_set))
    add_client(Accept(listenfd,...), &pool);
check_clients(&pool);
```

- `add_client · check_clients` 로 **연결 목록**을 직접 관리
- 끊기면 `FD_CLR` 로 감시 대상에서 제외
- 공유 자원을 **한 스레드만** 만지므로 락이 필요 없습니다

처리량 800

8.06

동시처리율 · 99.3ms

TASK 2

Thread-based

pthread

워커를 **미리 만들어 두고**, 마스터가 받은 연결을 **공유 버퍼**에 넣으면 워커가 꺼내 처리합니다.

구조

client 1
client 2
client 3

sbuf (공유 버퍼)
master가 넣고 worker가 꺼냄

worker
worker
... N개

여러 요청을 **실제로 동시에** 처리합니다 — 대신 공유 자원 보호가 필요합니다.

```
void sbuf_insert(sbuf_t *sp, int item) {
    P(&sp->slots); /* 빈칸 대기 */
    P(&sp->mutex); /* 버퍼 보호 */
    sp->buf[(++sp->rear) % sp->n] = item;
    V(&sp->mutex); V(&sp->items);
}
```

- 세마포어 **3개**(빈칸·아이템·상호배제)로 생산자·소비자 처리
- 세마포어를 **종목 노드마다** 뒀 다른 종목은 안 기다립니다
- 조회는 `readcnt` 로 세어 **첫 조회만** 쓰기를 막습니다

처리량 800

11.94

스레드 20 · 67.0ms

— 과제 04

malloc의 메모리 할당 방식을 직접 구현했습니다

수업에서 다룬 free list 네 가지 중 **Segregated** 방식을 선택해 구현

블록 구조 헤더·푸터에 크기와 할당 정보를 저장해 **앞뒤 블록의 상태**를 확인할 수 있도록 구성

HEADER
size | alloc

PAYLOAD – 사용자 영역
mm_malloc이 반환하는 포인터

FOOTER
size | alloc

FREE BLOCK
payload 영역에 prev · next 포인터 저장

...
next

8 byte align

PACK(size, alloc) 블록 크기를 **8 byte** 단위로 맞춰 하위 3비트 중 일부를 **할당 여부 표시**에 사용했습니다.

가용 블록 관리가용 블록의 **payload**에 prev-next 포인터를 저장해 **별도 공간 없이** free list를 구성했습니다.

01 크기별 Free List 구성

가용 블록을 **크기 구간별 10개 리스트**로 나눠 관리했습니다. 전체 힙을 순회하지 않고 **요청 크기에 해당하는 리스트부터** 확인합니다.

idx 0~3	~16 byte
idx 4	~32 byte ← 요청 크기 해당
idx 5	~64 byte
idx 6~9	128 byte 이상

getClass 요청 크기를 기준으로 사용할 **리스트의 index**를 결정

insertFree 같은 리스트 안에서 **크기 순서**에 맞춰 삽입

removeFree 할당할 블록을 제거하고 **앞뒤 노드 연결** 갱신

Segregated를 선택한 이유

Implicit은 **힙 전체**를 탐색하고, Explicit은 가용 블록만 확인하지만 **하나의 리스트**를 순회합니다.
Segregated는 크기별로 나눠 **탐색 범위를 줄일 수 있다**는 점을 선택 이유로 삼았습니다. 대신 여러 리스트를 관리하는 로직이 추가됩니다.

02 블록 분할과 병합

요청 크기보다 큰 블록을 찾으면 **필요한 크기만 할당**하고 남은 영역은 free list에 다시 등록했습니다. 해제 시에는 **인접한 가용 블록**을 병합했습니다.

```
/* place – 남은 공간이 충분하면 분할 */
if ((csize - asize) >= (2*DSIZE)) {
    PUT(HDRP(bp), PACK(asize, 1));
    PUT(FTRP(bp), PACK(asize, 1));
    /* 남은 블록을 free list에 등록 */
    insertFree(NEXT_BLPK(bp),
               csize - asize);
}
```

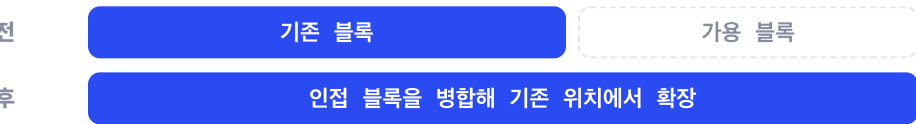
coalesce 해제 시 **앞·뒤 블록의 할당 상태**를 확인해 병합

extend_heap 적절한 가용 블록이 없으면 CHUNKSIZE만큼 힙 확장

find_fit 해당 크기 리스트부터 **더 큰 리스트까지** 순차 탐색

03 realloc 제자리 확장

더 큰 공간이 필요할 때 **인접한 가용 블록**을 먼저 **확인**해 병합했습니다. 새 블록을 할당하고 전체 데이터를 복사하는 경우를 줄였습니다.



앞쪽·뒤쪽·양쪽 세 경우를 모두 처리했으며, 앞쪽 블록과 병합하는 경우에만 memmove로 기존 데이터를 이동했습니다.

MDRIVER 성능 지수

강의 기본 구현 (Implicit)	55
Next-fit 확장	70
최종 구현 결과 — util 53 + thru 40	93

정확성 검사는 **전체 테스트 케이스**를 통과했고, 앞쪽 블록 병합을 추가한 뒤 일부 케이스의 utilization은 **99%**까지 확인했습니다.

기능은 정상적으로 동작했지만, 동시 요청에서 데이터 정합성 문제가 발생했습니다.

02 / 베틱기 기반 강의실 예약 시스템

Django · DRF · PostgreSQL · 베틱기 방식의 강의실 예약 백엔드

단일 요청에서는 문제가 없었지만, 같은 잔액과 같은 슬롯에 요청이 동시에 들어오면 데이터가 달라졌습니다.
동시 요청으로 문제를 재현한 뒤, 애플리케이션과 DB에서 각각 제어했습니다.

01

앱 레벨 — 비관적 락

베틱기 20건이 동시에 들어왔을 때 잔액에는 1건만 반영되던 문제를 행 잠금으로 해결

반영 1 → 20 / 20건

02

DB 레벨 — 부분 유니크 인덱스

같은 방·날짜에 여러 예약이 생성되는 문제를 DB 제약으로 차단

활성 예약 15 → 1 건

— 베팅 기반 강의실 예약 백엔드

동시 요청에서 잔액과 예약이 어긋났습니다

BACKGROUND

마일리지를 베팅해 강의실을 배정받는 예약 서비스입니다. 잔액과 예약 상태가 정확하지 않으면 배정 결과에도 영향을 줍니다.

잔액이 변경되는 지점

베팅	잔액 차감 + 베팅 기록 + 거래내역 생성
취소	경매 진행 중이면 참여자에게 환불
예약	같은 방·날짜 슬롯을 한 팀이 점유

예약 상태 6단계

- PENDING
- BIDS_OPEN
- BIDS_CLOSED
- CONFIRMED
- REJECTED
- CANCELLED

동시 요청 시 측정 결과 (BEFORE)

차감 반영

1건

베팅 20건

활성 예약

15건

정상값 1건

ROLE 백엔드 · 동시성 제어 · 테스트 설계

STACK Django · DRF · PostgreSQL 15 · pytest

PROBLEM

단일 요청에서는 통과했지만, 동시 요청에서 값이 달라졌습니다.

- 1

잔액 차감 누락

잔액

베팅 20건은 모두 생성됐지만 잔액에는 1건만 반영됐습니다. 같은 잔액을 읽은 요청들이 각자 저장하면서 이전 차감이 덮였습니다.
- 2

한 슬롯에 예약 15건

예약

모델에 유니크 제약이 없어 같은 방·날짜에 여러 예약이 동시에 생성됐습니다.
- 3

중복 환불 가능

취소 요청이 동시에 들어오면 두 요청 모두 진행 중 상태를 읽고 환불을 실행할 수 있었습니다.
- 4

검증과 변경 시점 분리

잔액 확인과 차감 사이에 다른 요청이 들어오면 검증에 쓴 값이 이미 이전 상태가 됩니다.

수정 전에 동시 요청으로 문제를 재현했습니다.

잔액이 어긋나는 과정

Lost Update

	요청 A · 1,000 차감	요청 B · 2,000 차감	DB 잔액
t1	READ 10000	—	10000
t2	—	READ 10000	10000
t3	계산 → 9000	계산 → 8000	10000
t4	WRITE 9000	—	9000
t5	—	WRITE 8000	8000

정상값은 7,000이지만, A의 차감 1,000이 B의 저장 과정에서 덮였습니다. Lost Update

기존 베팅 처리 reserve/views.py

BEFORE

```
# 검증이 트랜잭션 밖에서 수행됨
if request.user.mileage_balance < bet_amount:
    return Response(..., status=400)

with transaction.atomic():
    user.mileage_balance -= bet_amount
    user.save()
```

예약 전에 SELECT로 중복을 확인하면?

- A

SELECT — 중복 없음
- B

SELECT — 중복 없음
- A

INSERT
- B

INSERT — 둘 다 성공

TOCTOU — 조회와 저장 사이에 다른 요청이 들어올 수 있습니다.
transaction.atomic 만으로는 같은 행에 대한 동시 요청을 순차 처리할 수 없습니다.

— 해결 과정

앱과 DB에서 각각 동시 요청을 제어했습니다

1 비관적 락

앱 레벨

잔액 행을 잠근 뒤 검증과 차감 수행

선택

같은 잔액 행을 동시에 수정할 때 발생했습니다.

`select_for_update` 로 행을 잠근 뒤 다시 조회했습니다.베팅 처리 `reserve/views.py`

```
with transaction.atomic():
    user = User.objects.select_for_update() \
        .get(pk=request.user.pk)

    if user.mileage_balance < bet_amount:
        return Response(..., 400)

    user.mileage_balance -= bet_amount
    user.save()
```

t1	FOR UPDATE 획득	—
t2	검증 · 차감	대기(blocked)
t3	COMMIT · 락 해제	9000 읽음

B 요청은 A의 커밋 이후 갱신된 잔액을 기준으로 처리됩니다.

취소 · 환불

예약 행을 잠근 뒤 상태를 다시 확인해 동일 예약에 환불이 두 번 실행되지 않도록 했습니다.

2 부분 유니크 인덱스

DB 레벨

저장 시점에 중복 예약 차단

선택

앱에서 미리 조회하는 방식으로는 동시 INSERT를 막을 수 없습니다. DB가 저장 시점에 검사하도록 제약을 추가했습니다.

예약 모델 `reserve/models.py`

```
UniqueConstraint(
    fields=('room', 'reservation_date'),
    condition=~models.Q(status__in=(2, 5)),
    name='unique_active_reservation...',
)
```

- 조건 없이 적용하면 취소·거절된 슬롯도 다시 예약할 수 없어 활성 상태에만 제약
- `IntegrityError` 는 **409 Conflict**로 변환해 응답합니다
- 트랜잭션은 `atomic` 범위에서 롤백합니다

상태 전이

기능별로 분산돼 있던 상태 조건을 전이 테이블로 정리하고, 종료 상태에서는 추가 전이가 발생하지 않도록 했습니다.

제약의 적용 범위

DB 제약이므로 다른 API나 직접 실행한 SQL에서도 동일한 규칙이 적용됩니다.

3 적용 후 확인

적용 후

같은 조건으로 다시 측정

결과

수정 전 사용한 것과 동일한 조건으로 다시 동시 요청을 보내 결과를 비교했습니다.

잔액 차감

동시 20건

적용 전 20건 중 1건만 반영 · 오차 19,000

적용 후 20건 모두 반영 · 오차 0

중복 예약

동시 15건

적용 전 활성 예약 15건

적용 후 활성 예약 1건 · 나머지 409

- `TransactionTestCase` — 실제 트랜잭션 동작이 필요한 테스트에 사용
- `threading.Barrier` — 여러 스레드의 요청 시작 시점을 맞춰 동시 호출

회귀 방지

동일한 동시 요청 조건을 테스트로 남겨 이후 코드 변경에서도 계속 확인할 수 있도록 했습니다.

— 결과와 회고

동시 요청에서도 데이터가 일관되게 처리됩니다

RESULT · 동일 조건 재측정

잔액 차감 반영 — 동시 20건 베팅

4 → 20 / 20건

생성된 베팅 20건이 모두 잔액에 반영됩니다.

활성 예약 — 동시 15건 요청

15 → 1 건

1건만 201, 나머지 14건은 409로 거절됩니다.
취소된 슬롯은 다시 예약할 수 있습니다.

중복 환불 차단

락 + 상태 재확인

취소 요청이 겹치면 두 번째 요청은 이미 취소된 상태로 처리됩니다.

잘못된 상태 전이 차단

전이 테이블

취소된 예약을 다시 확정하는 등 허용하지 않은 상태 변경을 막았습니다.

측정 조건

DB

PostgreSQL — 행 단위 락 사용

테스트

TransactionTestCase — 트랜잭션 경합 재현

동시성

threading.Barrier — 요청 시작 시점 동기화

WHY · 비관적 락을 선택한 이유

비관적 락을 선택한 이유

세 방식 모두 Lost Update를 막을 수 있지만, 동일한 잔액에 요청이 집중되는 상황과 검증 로직을 함께 고려해 비관적 락을 사용했습니다.

방식	특징	선택
비관적 락 select_for_update	같은 행에 경합이 발생하면 순차 처리됩니다. 별도의 충돌 재시도 로직이 필요하지 않습니다.	채택
원자적 연산 F()	단순 증감에는 가볍지만 여러 검증 조건을 함께 처리하기에는 제약이 있습니다.	대안
낙관적 락 version	충돌이 드문 환경에 적합합니다. 충돌 시 재시도 처리가 필요합니다.	대안

동일한 잔액에 요청이 집중되는 구조와 검증 조건을 고려해 비관적 락을 선택했습니다.

체감이 아니라 **측정값으로** 병목을 확인했습니다.

03 / 학교 축제 서비스 Cardinal

React · TypeScript · Mapbox 기반 축제 지도 페이지 성능 개선 · 프론트엔드

사용자가 자주 이용하는 지도 페이지에서 **리스트를 여닫을 때** 심한 **프레임 드롭**이 발생했습니다.

가설을 세우고 측정한 뒤, 효과가 부족하면 다시 원인을 분석하며 **병목 지점을** 좁혀갔습니다.

담당 페이지

지도

부스

굿즈

GNB

백오피스

— 학교 축제 지도 페이지

마커·리스트를 열 때
끊기는 지도 화면

BACKGROUND

동아리에서 개발한 학교 축제 웹 서비스로, Mapbox 지도 위에 9개 카테고리 마커와 부스 리스트를 구현했습니다.

지도 페이지 주요 기능

카테고리	선택 시 해당 마커만 지도에 표시
마커 클릭	주점·푸드트럭 등 부스 리스트를 드로어로 표시
드로어	높이를 20%→80%로 조절하는 인터랙션

INTERACTION (INP)

약 97% 단축

1130 → 343 → 34ms

ROLE 프론트엔드 · 지도 페이지 · 성능 측정·개선

STACK React · TypeScript · Mapbox · framer-motion

PROBLEM

기능은 동작했지만, 리스트를 여닫을 때 화면이 끊겼습니다.

- 1

드로어 애니메이션 프레임 드롭

리스트를 열고 닫는 과정에서 심한 프레임 드롭이 발생했습니다.
- 2

사용 빈도가 높은 화면

지도와 부스 목록은 사용자가 반복해서 조작하는 화면이라 지연이 바로 체감됐습니다.
- 3

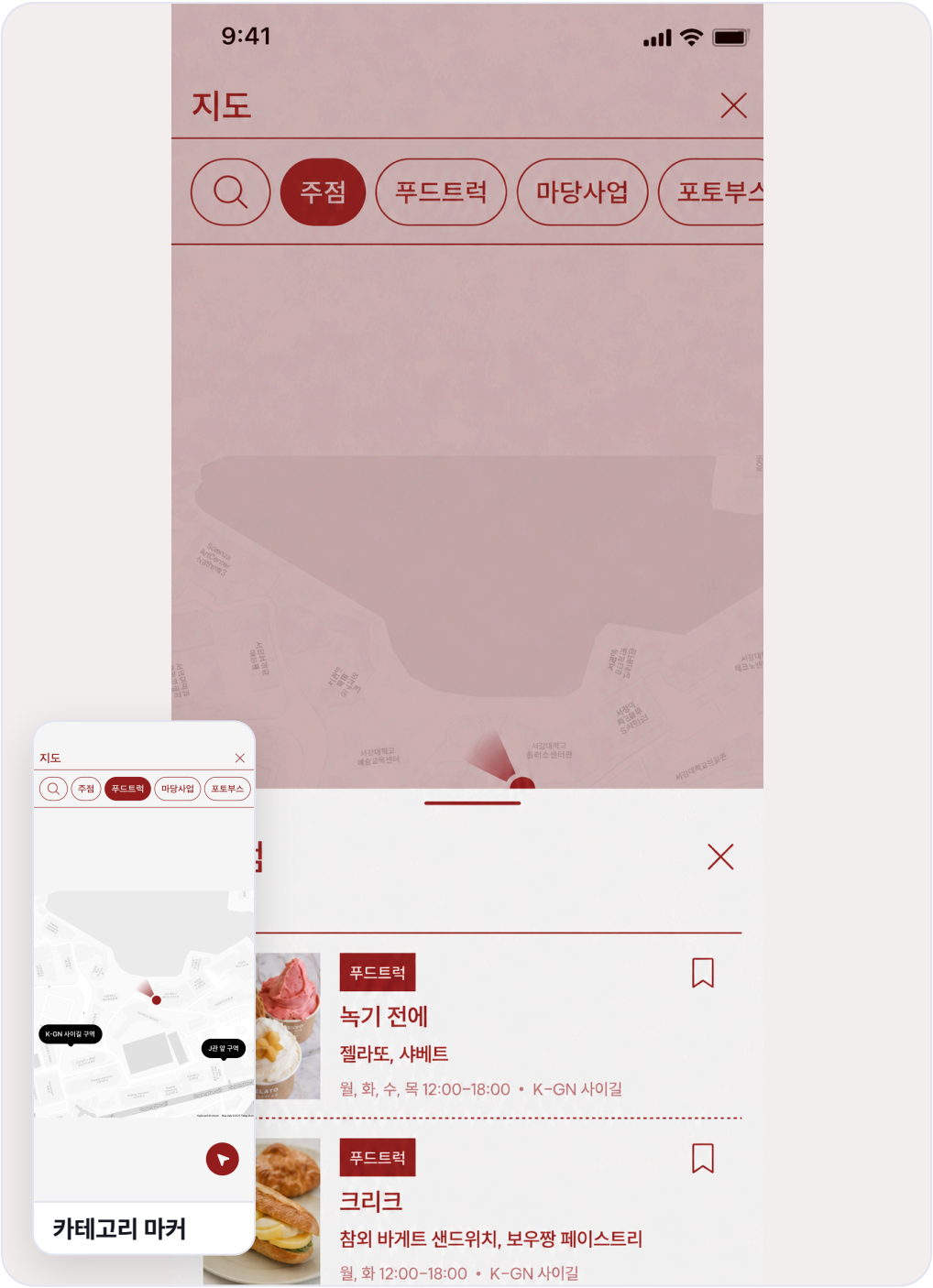
일반 지표만으로 원인 확인 어려움

Lighthouse의 LCP·TBT만으로는 해당 인터랙션 지연을 설명하기 어려웠습니다.

Performance 탭과 Clarity를 추가로 확인하며 병목을 좁혔습니다.

문제가 발생한 화면

마커 → 부스 리스트 드로어



마커 클릭 → 부스 리스트 드로어 · 여닫을 때 프레임 드롭 발생

— 병목 추적

병목 원인을 단계별로 확인했습니다

1 Lazy Load

개선 제한

이미지 요청 시점 늦추기

가설

한 번에 많은 이미지를 불러오는 것이 원인이라고 보고 **lazy loading**을 먼저 적용했습니다.

리스트의 썸네일

```
<img
  src={src}
  loading="lazy"
  decoding="async"
  fetchPriority="low"
/> // 펼치면 전체 항목은 DOM에 마운트
```

- 기대 — 화면 밖 이미지 요청을 미뤄 초기 처리량 감소
- 실측 — 체감은 일부 개선됐지만 지표 변화는 제한적

확인

lazy 는 요청 시점을 조절하지만, 드로어를 열 때 **항목 전체가 DOM에 생성**되는 구조는 그대로였습니다.

2 IntersectionObserver

부분 개선

화면 근처 이미지만 렌더링

시도

IntersectionObserver로 화면 근처에 들어온 항목만 를 생성하도록 변경했습니다.

LazyImage.tsx

```
const io = new IntersectionObserver((e) => {
  if (e[0].isIntersecting) {
    setInView(true);
    io.unobserve(e[0].target);
  }
}, { root, rootMargin, threshold });

{inView && !error && <img src={src} ... />}
```

- rootMargin 으로 화면 진입 전에 미리 로드
- 감지 후 unobserve 로 불필요한 재관찰 방지
- 재사용 컴포넌트에 memo + skeleton 적용

INP 1130 → 343MS

열 때는 개선됐지만 **닫을 때는 여전히 프레임 드롭**이 발생해, 이미지 개수 외의 **처리 비용**을 확인했습니다.

3 원본 이미지 해상도

주요 병목

Network 탭에서 이미지 크기 확인

발견

이미지 **응답 시간이 길어** 원본 파일을 확인한 결과, 실제 표시 크기에 비해 **해상도가 지나치게 컸습니다.**

6000×4000 약 2,400만 픽셀

- 표시 크기 **600×400** 대비 픽셀 수 **약 100배**
- 화면에서 작게 표시돼도 **고해상도 원본**을 처리한 뒤 축소

개선안

측정 결과를 정리해 업로드 시 **AWS Lambda로 이미지를 자동 리사이즈**하는 방식을 백엔드에 제안했습니다.

병목 확인

고해상도 이미지 처리 비용이 드로어 애니메이션과 겹치면서 응답 지연과 프레임 드롭이 발생했습니다.

— 개선과 결과

프론트와 이미지 처리 구조를 함께 개선했습니다

HOW · 처리 대상 수와 이미지 크기를 각각 줄였습니다

FRONTEND · 담당

1130 → 343 ms

렌더링 대상 수 감소

IntersectionObserver로 화면 근처 항목만 를 생성해 한 번에 처리하는 이미지 수를 줄였습니다.

프론트 개선 이후에도 지연이 남았습니다.

화면에 보이는 이미지 자체가 6000×4000 원본이어서 이미지 처리 비용이 계속 발생했습니다.

BACKEND · 제안

343 → 34 ms

이미지 리사이즈 제안

측정 결과와 원본 이미지 크기를 공유하고, 업로드 시 AWS Lambda에서 표시 크기에 맞게 자동 리사이즈하도록 제안했습니다.

프론트에서는 처리할 이미지 수를 줄이고, 백엔드에서는 이미지 한 장의 처리 비용을 줄이는 방식으로 개선했습니다.

RESULT

INP — 마커 클릭 후 응답

1130 → 34 ms

처리 대상 수와 이미지 크기를 함께 줄여 약 97% 단축

• 프레임 드롭 개선

반복 드롭 → 정상 동작

긴 리스트를 닫을 때도 이전보다 끊김 없이 동작했습니다.

• 이미지 처리량 감소

2,400만 → 24만 px

표시 크기에 맞게 리사이즈해 이미지당 픽셀 수를 약 1/100로 축소했습니다.

• 네트워크 전송량 감소

원본 이미지 전송량 ↓

화면에 필요한 이미지만 표시 크기에 맞는 해상도로 받아 전송량도 줄였습니다.

• 사용자 행동 확인

MS Clarity

세션 녹화에서 개선 전후를 비교하고 반복 클릭 감소를 확인했습니다.

측정값을 기준으로 원인을 좁히고, 프론트와 이미지 처리 구조를 함께 개선했습니다.

THANK YOU

읽어주셔서 감사합니다

궁금한 점이나 함께 이야기하고 싶은 부분이 있다면
언제든 편하게 연락 주세요.

EMAIL

shu07002@gmail.com

PHONE

+82 10-3997-5917

GITHUB

github.com/shu07002

BLOG

shu07002.tistory.com